

EXAMEN MÓVILES

Nombres: Carlos Alberto

Apellidos: Candhuiri Ibarancea

Código: 22200175

Fecha: 20/06/26

2. Análisis de arquitectura y concurrencia

1. El Patrón Repositorio como SSOT

El repositorio se considera SSOT o única fuente de verdad porque es el único componente autorizado para dar los datos a la aplicación, la UI no accede directamente a la base de datos ni a la API, sino que obtiene la información únicamente a través del repositorio. Además, el repositorio no actúa como una "fachada" porque oculta la complejidad de las distintas fuentes de datos y ofrece una interfaz simple para el resto de la aplicación.

Coordinación entre Room y Retrofit:

- Mientras que Room almacena los datos localmente, retrofit los obtiene desde un servidor remoto. El repositorio decide cuándo consultar la base de datos local o cuándo realizar una petición a la API, una vez obtenidos los datos remotos el repositorio los guarda en Room sin conocer el origen real. De esta forma la UI permanece desacoplada de las fuentes de datos y sólo interactúa con el repositorio.

2: Corrutinas y Structured Concurrency

View Model Scope es un alcance de corrutinas asociado al ciclo de vida del View Model, permite ejecutar tareas asíncronas de forma segura y evita fugas de memoria.

Si el usuario cierra la pantalla antes de que termine una petición de red, el View Model se destruye y el View Model Scope se cancela automáticamente, como consecuencia, todas las corrutinas hijas también se cancelan gracias a Structured Concurrency.

3: Flow (Cold) vs. StateFlow (Hot)

Un Flow es un flujo frío (Cold Flow), por lo que sólo comienza a emitir datos cuando existe un colector. Cada nuevo colector ejecuta nuevamente el flujo.

Un StateFlow es un flujo caliente (Hot Flow), permanece activo y conserva siempre el último valor emitido.

Se refiere StateFlow para la UI porque mantiene el estado actual de la aplicación y permite que nuevos observadores reciban inmediatamente el último valor disponible.

4: Inyección de dependencias (DI) manual

La DI manual consiste en crear las dependencias dentro de una clase que hereda de *Application* y compartirlas con el resto de la aplicación mediante referencias comunes.

La palabra reservada `by lazy` realiza una inicialización perezosa, crea el objeto se crea únicamente cuando se utiliza por primera vez, esto optimiza tanto la memoria como los recursos, además de garantizar una única instancia compartida de componentes como la base de datos o el repositorio.

II. Sección de completar.

1. El flujo de datos que emite automáticamente cada vez que una tabla Room cambia se implementa usando el tipo Flow

2: El operador `Dispatchers.Default` permite ejecutar tareas pesadas fuera del *Main Thread*, específicamente optimizado para tareas de CPU como el parseo masivo de datos.

3: Para que un servicio en primer plano de ubicación sea válido en Android 10+, debe declararse en el manifiesto con el tipo location

4: La función `onCreate()` de la clase *Application* es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un "cast"

5: El componente que permite transformar una API basada en callbacks antiguos en una función suspendible moderna se llama `suspend Coroutine`.

6: El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente `AndroidManifest.xml`.

7: En la arquitectura ACBP, la capa que actúa como puente estándar entre el *framework* y los controladores se denomina HAL.

8: El operador de *Flow* que permite combinar múltiples fuentes (como GPS y sensores) para emitir un estado unificado es `combine`.

III. Desarrollo de Código y Lógica Personalizada

1. Repositorio y DI

• Clase Usuario Repository

```
class UsuarioRepository {  
    private var usuarioDao: UsuarioDao  
}
```

• Instanciación en Application usando by lazy

```
class MyApplication : Application() {  
    private var database by lazy {  
        AppDatabase.getInstance(this)  
    }  
    var usuarioRepository: UsuarioRepository by lazy {  
        UsuarioRepository(database.usuarioDao())  
    }  
}
```

2. Lógica Dinámica de Identidad

a) Data Class

```
@data class Usuario {  
    var nombre: String?,  
    var apellidos: String?,  
    var edad: Int?  
}
```

b) Objeto con datos reales

```
var usuario = Usuario(  
    nombre = "Carlos Alberto",  
    apellidos = "Canaquiri Huapana",  
    edad = 22  
)
```

c) Calcular Puntaje de Identidad

```
var puntaje =  
    (usuario.nombre?.length ?: 0) +  
    (usuario.apellidos?.length ?: 0)
```

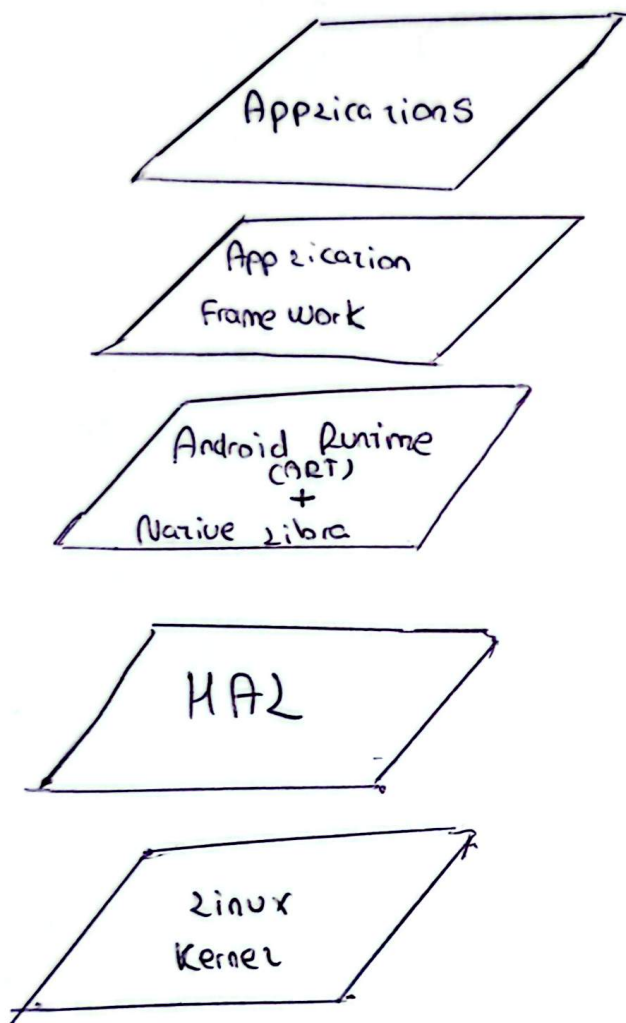
d) Imprimir el mensaje

```
println {  
    "Usuario: {usuario.nombre} {usuario.apellidos},  
    su puntaje de identidad es: {puntaje}"  
}
```

IV. Arquitectura del AOSP Completa

1: Gráfico

2: Identificación verbal



Es la capa superior donde se ejecutan las aplicaciones del usuario y del sistema.

Existen apps como Mensajes, apps de teléfono, Cámara, etc.

Proporciona los servicios y APIs que usan las aplicaciones para interactuar con el SO. Incluye componentes como Activity Manager, Window Manager, etc.

Ejecuta las aplicaciones android. Incluye ART (compilación, memoria y garbage collection) y bibliotecas nativas como SQLite y OpenGL.

Actúa como capa de abstracción entre el Framework Android y el hardware del dispositivo. Define interfaces estándar para acceder a componentes como cámara o GPS sin depender de controladores específicos de cada fabricante.

Capa base del sistema operativo, gestiona hardware, memoria, procesos, drivers, seguridad y comunicación con el dispositivo.